Product Requirements Document: Agentic MicroVM Sandboxes

1. Overview and Objectives

Objective: Implement a secure, instant-boot computing environment (Sandbox) for AI agents built on the rustakka/atomr-agents framework. **Context:** Inspired by LangSmith's Sandbox architecture, agents require the ability to execute untrusted code (Python, Bash, JavaScript, Rust), manipulate real filespaces, and persist state without exposing the host infrastructure. Containers (Docker) share a kernel and are insufficient for untrusted AI-generated code. **Solution:** Hardware-virtualized microVMs using **AWS Firecracker** (via KVM), orchestrated by Rust. The system supports execution topologies from local dev loops to bare-metal clusters, exposing composable compute to atomr-agents.

2. Deployment Topologies

To support developers and production environments, the Sandbox architecture utilizes a pluggable backend strategy. The SandboxTool can seamlessly target three distinct operational tiers:

2.1 Tier 1: Local Developer Machine

- **Target:** Laptops and local workstations during the dev loop.
- **Linux:** Uses native KVM directly via /dev/kvm.
- **macOS / Windows:** Uses a lightweight hypervisor wrapper (like Lima, Multipass, or WSL2) to expose KVM to the local Rust orchestrator.
- **Degradation:** If KVM is completely unavailable, the local orchestrator falls back to a restricted Docker container purely for local testing, issuing an "Insecure Dev Mode" warning.

2.2 Tier 2: Dedicated Host (Single Box)

- **Target:** A single bare-metal server optimized for high-throughput AI execution.
- **Architecture:** The atomr-agents framework and the Sandbox Orchestrator run on the same machine.
- **Storage:** Utilizes a local fast NVMe drive with Btrfs or OverlayFS for lightning-fast copy-on-write snapshot forks.

2.3 Tier 3: Clustered (Horizontal Scale)

- **Target:** Production deployment handling thousands of concurrent agents across multiple bare-metal nodes (via Kubernetes or Nomad).
- **Mechanism:** Bare-metal worker nodes run a Rust DaemonSet handling local Firecracker instances. A Control Plane load-balances .create_sandbox() requests. Communication

(virtio-vsock) is tunneled securely over gRPC/mTLS.

3. System Architecture

3.1 Host Layer (Control Plane & Abstraction)

- **Agent Framework (atomr-agents)**: Runs the primary LLM pipeline, memory state, and tool-call fan-out.
- **Sandbox Orchestrator Client**: Abstracts the routing (Local daemon vs. Remote gRPC Control Plane).
- **Snapshot Pool Manager**: Maintains pre-warmed, paused Firecracker microVM snapshots on target nodes.

3.2 Guest Layer (Execution Environment & Toolchains)

- **Minimal Guest OS**: A stripped-down Linux kernel optimized for Firecracker.
- **Guest Agent daemon**: A statically compiled Rust binary running as the init process (PID 1), listening exclusively on AF_VSOCK.
- **Runtime Profiles (RootFS Base Images)**: The root filesystem is built dynamically or pre-provisioned based on the toolchain the agent needs.

3.3 The Bridge (virtio-vsock & Tunneling)

- No standard TCP/IP networking is exposed to the guest by default.
- Communication is strictly handled via virtio-vsock. In Clustered Mode, this is tunneled over the network via the Node Agent.

4. Environment Profiles & Toolchains

Because different AI tasks require different toolchains, and because full compiler toolchains (like Rust) heavily impact memory and rootfs size, the Sandbox supports explicit configuration profiles.

4.1 Supported Config Options

The orchestrator maintains pre-warmed snapshot pools for the following environment profiles:

1. **PythonOnly**: Contains Python 3.11+, pip, and standard data science libraries. (Lightweight, fast).
2. **NpmOnly**: Contains Node.js 20+ and npm/yarn/pnpm. (Frontend/JS-agent focused).
3. **RustOnly**: Contains rustc, cargo, rustup, and build-essential. (Heavyweight, requires higher RAM budget for compilation).
4. **PythonAndNpm**: Contains both Python and Node.js environments.
5. **FullStack**: Contains Python, NPM, and the complete Rust compilation toolchain.

4.2 Resource Allocation Impacts

- Interpreted languages (Python/NPM) can comfortably run in 512MB RAM / 1 vCPU.

- **Rust Compilation Constraint**: Due to the memory-intensive nature of rustc and LLVM, any profile including Rust (RustOnly, FullStack) will automatically enforce a minimum AgentBudget of **2GB RAM and 2 vCPUs** in the microVM provisioning request.

5. Integration with rustakka/atomr-agents

5.1 Rust Host Integration

The SandboxTool configuration accepts the target backend and the requested toolchain profile.

```
use atomr_agents::prelude::*;
use atomr_agents::tool::{RichTool, ToolReturn, ToolDescriptor};

pub enum SandboxBackend {
    LocalDevMode,
    DedicatedHost { socket_path: String },
    Cluster { grpc_endpoint: String, token: String },
}

pub enum SandboxProfile {
    PythonOnly,
    NpmOnly,
    RustOnly,
    PythonAndNpm,
    FullStack,
}

pub struct SandboxTool {
    backend: SandboxBackend,
    profile: SandboxProfile,
}

#[async_trait]
impl RichTool for SandboxTool {
    fn descriptor(&self) -> ToolDescriptor {
        ToolDescriptor {
            name: "execute_in_sandbox".into(),
            description: "Executes code in a secure microVM.".into(),
            // Exposes parameters like `language`, `code`,
`dependencies`
        }
    }

    async fn call(&self, input: Value, ctx: &AgentContext) ->
Result<ToolReturn, ToolError> {
        // 1. Resolve backend & Request specific Profile from Snapshot
Pool
        // 2. Adjust vCPU/RAM budgets if `RustOnly` or `FullStack` is
```

```
requested
        // 3. Serialize request & send over tunneled AF_VSOCK
        // 4. Await stdout/stderr/compilation errors
        // 5. Return execution results
    }
}
```

5.2 Python (PyO3) Bindings

Python developers can define which profile their agent needs when provisioning the sandbox.

```
from atomr_agents.sandbox import SandboxClient, SandboxConfig,
SandboxProfile

# Configure for a FullStack environment to allow the agent to write
and compile Rust
config =
SandboxConfig.cluster(endpoint="grpc://sandbox.internal:50051")
sandbox = SandboxClient.create(
    profile=SandboxProfile.FullStack,
    config=config
)

@tool
def write_and_run_rust(source_code: str) -> str:
    """Writes Rust code to main.rs, compiles it with cargo, and runs
the binary."""
    # Write the file via vsock
    sandbox.write_file("src/main.rs", source_code)
    # Compile and execute
    return sandbox.run("cargo run --release")
```

6. Functional Requirements

6.1 VM Lifecycle Management

1. **Cold Start**: The node orchestrator must boot a microVM (from any profile) within 200ms.
2. **Warm Start (Forking)**: Support pausing a VM, taking a memory snapshot, and resuming multiple clones to back LangGraph/atomr branch states.
3. **Cluster Scheduling**: The Control Plane must bin-pack VMs based on the SandboxProfile requirements (e.g., allocating FullStack VMs to nodes with higher memory availability).

6.2 State and Observability

1. **Channeled State**: The Sandbox tool must report execution boundaries (start, end, error) to the atomr-agents EventBus.

2. **Telemetry**: Execution metrics, specifically guest-memory spikes during rustc compilation, must be logged via LangSmithTracer.

7. Non-Functional Requirements

7.1 Security & Isolation

- **Hardware Virtualization**: The boundary *must* be KVM-backed via Firecracker jailer.
- **Auth Proxying**: If the guest needs internet access (e.g., cargo build, npm install), outbound traffic flows through an egress proxy injecting credentials.

7.2 Performance Target

- **Snapshot Resume Time**: < 50ms (node-local).
- **Clustered Network Overhead**: < 10ms added latency per command for gRPC -> vsock tunneling.
- **Rust Compilation**: The underlying host NVMe drives must provide sufficient IOPS to ensure target/ directory writes during cargo build do not bottleneck execution.

8. Implementation Phases

Phase 1: Foundation (Local & Dedicated)
- Implement firecracker-rs-sdk wrappers & Guest Agent vsock daemon.

Phase 2: RootFS & Toolchain Image Bakery
- Create a CI/CD pipeline (using mkosi or standard ext4 creation tools) to automatically build and maintain the 5 specific SandboxProfile base images (PythonOnly, NpmOnly, RustOnly, PythonAndNpm, FullStack).

Phase 3: Framework Integration (atomr-agents)
- Wrap the communication layer into a SandboxTool.
- Map the memory/CPU AgentBudgets logic to scale dynamically based on the requested toolchain.
- Add PyO3 bindings for SandboxProfile.

Phase 4: Snapshot & Cluster Sync
- Map atomr-agents fork-with-edit capabilities to Firecracker memory snapshots.
- Develop the gRPC Control Plane for routing tiered sandbox requests.